Software Project Lab-01

Static Code Energy Ananlyzer- Operation Level Energy Profiling Before Compilation

By Nafis Ahmed
(BSSE 1621)

Supervised by: Dr. B M Mainul Hossain
Professor, IIT, University of Dhaka

Project Overview & Motivation

Problem

Modern compilers optimize for speed and memory, but energy consumption is largely ignored. This is critical for embedded systems, IoT devices, and mobile applications where battery life matters.

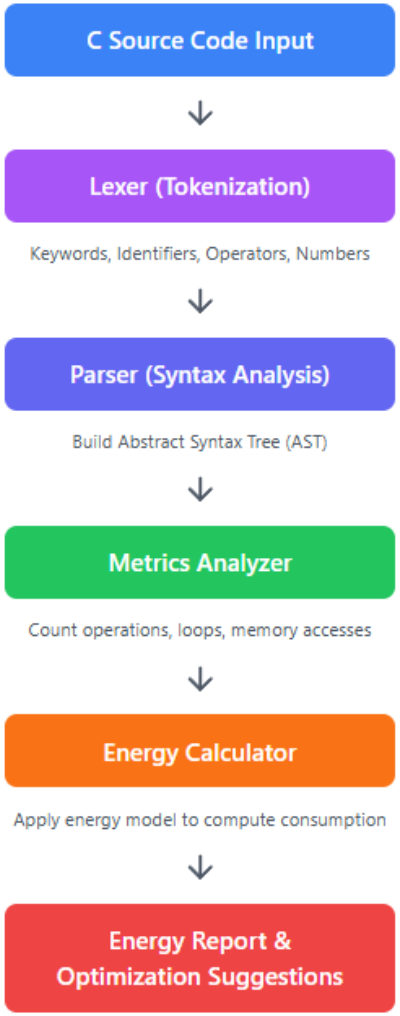
Solution

Build a static analysis tool using compiler techniques (lexical analysis, parsing, AST) to estimate energy consumption by analyzing operations, loops, memory accesses, and function calls in C programs.

Objective

- Analyze C source code and compute energy metrics per function
- Identify energy-intensive code sections (hotspots)
- Help developers write energy-efficient code
- Provide foundation for energy-aware compilation

Project Architecture & Workflow



Completed Components - Implementation Details

1. Lexer Module (Complete)

Tokenizes C code into atoms. Handles 34 keywords (int, for, while, return, etc.), all operators (+, -, *, /, &&, ||, &, |, ^, ==, !=, etc.), string/char literals, comments, tracks line/column for errors.

```
int sum = a + b * 2; → KEYWORD(int) IDENTIFIER(sum) OPERATOR(=) IDENTIFIER(a) OPERATOR(+) IDENTIFIER(b) OPERATOR(*) NUMBER(2)
PUNCTUATION(;)
```

2. AST Data Structures (Complete)

Defines complete AST node types: Expressions (Identifier, Number, String, UnaryOp, BinaryOp, FunctionCall, ArrayAccess), Statements (Declaration, Assignment, Return, If/Else, While, For, Break, Continue, Block). Represents program structure hierarchically.

3. Energy Model (Complete)

Energy coefficients: Arithmetic (0.3), Comparison (0.5), Logical (0.2), I/O (1.0), Memory Access (0.6), Allocation (2.0), Function Call (0.8), LOC (0.05). Special handling: loop operations × iteration count, recursion × depth factor (10).

```
Energy = Sum(operation_count × energy_coefficient) + loop_factor + recursion_penalty
```

4. Parser (Partial)

Basic structure defined (peek, get, match functions). Core parsing logic not yet implemented.

Lexer Implementation

Key Features:

✓ Recognizes 34 C keywords (int, for, while, return, etc.)

✓ Handles single-line (//) and multi-line (/* */) comments

✓ Supports operators: arithmetic (+, -, *, /), logical (&&, ||), bitwise (&, |, ^)

✓ Tracks line and column numbers for error reporting

✓ Processes strings, characters, and numeric literals (decimal, hex)

```
// Example Input
int sum = a + b * 2;

// Tokens Generated:
KEYWORD(int) IDENTIFIER(sum) OPERATOR(=)
IDENTIFIER(a) OPERATOR(+) IDENTIFIER(b)
OPERATOR(*) NUMBER(2) PUNCTUATION(;)
```

AST Structure

Expression Types

- Identifier
- Number, String
- Unary Operations (-x, !x)
- Binary Operations (x + y)
- Function Calls
- Array Indexing

Statement Types

- Declarations (int x = 5)
- If/Else
- While Loops
- For Loops
- Return Statements
- Block Statements

```
// Example: Binary Operation AST
BinaryOpExpression:
op: "+"
left: IdentifierExpression("a")
right: NumberExpression("5")
```

Energy Model Design

Energy cost per operation (arbitrary units):

Comparisons: 0.5	**Arithmetic:** 0.3
I/O Operations: 1.0	**Memory Alloc:** 2.0
Function Call: 0.8	**Memory Access:** 0.6

Special Handling:

- Loop operations multiplied by estimated iteration count
- Recursive functions use depth factor (default: 10)
- Base energy cost per line of code: 0.05

Remaining Work & Implementation Strategy

1. Complete Parser Implementation

- parseExpression(): Recursive descent with operator precedence (handles +, -, *, /, %, ==, !=, operators, function calls, array indexing)
- parseStatement(): Parse declarations (int x = 5), if/else, while, for, return, blocks
- parseFunctions(): Extract function definitions with return type, name, parameters, body
- Error handling: Syntax error detection with line/column reporting

2. Build Metrics Analyzer

- AST Traversal: Visitor pattern to walk through all nodes
- Operation Counting: Count arithmetic, comparisons, logical, bitwise, I/O, memory operations
- Loop Analysis: Detect for/while loops, estimate iteration count using simple static analysis
- Recursion Detection: Identify functions calling themselves
- Output: FunctionMetrics structure with all counts

3. Energy Calculator & Report Generator

- Apply energy model to FunctionMetrics: Energy = Sum(count × coefficient)
- Generate report: Function name, LOC, total energy, breakdown by operation type
- Identify hotspots: Functions with highest energy consumption
- Suggest optimizations: Replace expensive operations, reduce I/O, optimize loops

Expected Outcomes & Optional Extensions

Expected Deliverables

✓ Working tool that analyzes C programs and reports energy consumption per function
✓ Detailed report showing operation counts, energy estimates, and hotspot identification
✓ Validation against test programs demonstrating correctness
✓ Foundation for future energy-aware compiler optimizations

Optional Extensions (If Time Permits)

Advanced Loop Analysis: Symbolic execution for better iteration estimates

Interprocedural Analysis: Track energy across function call chains

Optimization Engine: Automated suggestions for loop unrolling, strength reduction

GUI Dashboard: Interactive visualization with energy heatmaps

Hardware Validation: Compare with actual energy measurements

Multiple Languages: Extend to C++, Java

Impact: Help developers write energy-efficient code for embedded systems, IoT devices, and mobile applications where battery life is critical

Thank You